

DIARIO DEL PROGETTO

MEMBRI:

- ANTONELLA BELEN, CANDIA, Matr: 0001097853
- AURORA, CORTECCHIA, Matr: 0001069738
- ALESSIA, DIETTI, Matr: 0001097971
- LORENZO, STABILE, Matr: 0001069691

INDICE

- Sprint 0 , dal 03/11 al 09/11	3
- Sprint 1 , dal 10/11 al 16/11	7
Product Owner: Lorenzo	
Scrum Master: Alessia (approva la pull request)	
Developer 1: Antonella (pull request)	
Developer 2: Aurora	
- Sprint 2 , dal 17/11 al 23/11	11
Product Owner: Aurora	
Scrum master: Lorenzo (approva la pull request)	
Developer 1 alessia (pull request)	
Developer 2 antonella	
- Sprint 3 , dal 24/11 al 30/11	15
Product Owner: Antonella	
Scrum Master: Aurora(approva la pull request)	
Developer 1: Lorenzo (pull request)	
Developer 2: Alessia	
- Sprint 4 , dal 01/12 al 07/12	22
Product Owner: Alessia	
Scrum Master: Antonella(approva la pull request)	
Developer 1: Aurora (pull request)	
Developer 2: Lorenzo	
- Sprint 5 , dal 08/12 al 14/12	27
Product Owner: Lorenzo	
Scrum Master: Alessia (approva la pull request)	
Developer 1: Antonella (pull request)	
Developer 2: Aurora	
- Burn down chart	33
- Riflessione finale	34

Sprint 0 — Preparazione

Periodo: 03/11/2025 – 09/11/2025

Lo Sprint 0 ha rappresentato la fase preparatoria del progetto *Nota Bene*, ed è stato dedicato alla definizione della visione iniziale del prodotto, alla creazione dei materiali di analisi necessari e all'organizzazione del gruppo di lavoro.

Lo scopo principale di questo sprint non era la produzione di funzionalità implementative, ma la costruzione di una **base concettuale, organizzativa e tecnica chiara, condivisa e stabile**, su cui poter sviluppare in modo coerente ed efficace tutti gli sprint successivi.

In questa fase preliminare, il team ha lavorato sull'identificazione dei requisiti fondamentali del sistema, sulla modellazione del dominio applicativo, sulla definizione dei casi d'uso principali e sulla scelta consapevole dello stack tecnologico. Parallelamente, sono state impostate la struttura del repository GitHub e le regole di collaborazione interne.

Lo Sprint 0 ha, quindi, garantito una visione comune del progetto e una pianificazione iniziale solida, riducendo fin da subito il rischio di ambiguità, incomprensioni o disallineamenti nelle fasi successive dello sviluppo.

Scelta dello stack tecnologico

Lo stack selezionato è stato il seguente:

- **Backend**
 - **Spring Boot 3.3.2:** framework maturo, modulare e ampiamente utilizzato, scelto per facilitare lo sviluppo rapido e strutturato di API REST
 - **Java 17:** versione LTS, compatibile con Spring Boot, stabile e performante, adatta a progetti di medio-lungo termine
 - **Maven:** utilizzato come sistema di build e gestione delle dipendenze, per garantire riproducibilità e controllo delle versioni
 - **MapDB:** database embedded leggero, scelto per supportare una persistenza semplice nelle fasi iniziali e per consentire un'evoluzione incrementale del sistema
 - **JUnit 5:** framework di testing automatico, obbligatorio per il progetto e fondamentale per garantire qualità e affidabilità del codice
- **Frontend**
 - **HTML, CSS, JavaScript** nella loro forma più semplice, utilizzati per realizzare prototipi funzionali delle schermate principali
 - Utilizzo dell'API **fetch()** per la comunicazione asincrona con il backend tramite chiamate REST
- **Strumenti di supporto**
 - **GitHub** per il versionamento del codice, la gestione delle Pull Request e il coordinamento del lavoro
 - **GitHub Projects** per la gestione del backlog e la pianificazione degli sprint tramite board Kanban

- **WhatsApp** come canale diretto e immediato di comunicazione tra i membri del team
- **Agenda/To-Do List condivisa** per annotare idee, proposte, attività completate e aspetti da migliorare
- **Microsoft Teams** per riunioni di sprint planning, allineamenti tecnici e sessioni di revisione/dimostrazione quando non era possibile incontrarsi in presenza.

Le motivazioni alla base di queste scelte tecnologiche sono state approfondite e documentate nel **Manuale dello Sviluppatore**, redatto in parallelo allo Sprint 0.

Setup del repository GitHub

Durante lo Sprint 0 è stato impostato l'intero ambiente di lavoro su GitHub, con particolare attenzione all'organizzazione e alle buone pratiche di versionamento:

- creazione della repository principale del progetto;
- configurazione della board Kanban con le colonne *Product Backlog*, *Sprint Backlog*, *In Progress* e *Done*;
- abilitazione e adozione delle **Pull Request** come meccanismo di integrazione controllata;
- definizione di convenzioni minime per commit e naming, al fine di garantire uniformità e tracciabilità.

In merito alla strategia di branching, il team ha definito una regola interna consistente nell'utilizzo di **un branch dedicato per ciascuno sprint**, evitando la proliferazione di branch separati per singoli Use Case. Tale scelta è stata motivata dall'esigenza di adottare un workflow **più lineare e gestibile** rispetto a soluzioni più granulari, riducendo il rischio di errori procedurali e conflitti di merge in un contesto di apprendimento e sviluppo collaborativo.

Inoltre, è stata concordata fin dall'inizio una **cadenza ricorrente** per l'apertura e la revisione delle Pull Request (generalmente **venerdì/sabato**, salvo esigenze particolari), al fine di concentrare le attività di integrazione, testing e review in finestre temporali dedicate e prevedibili.

Questo setup iniziale ha fornito una struttura di lavoro ordinata e condivisa, contribuendo a rendere più stabile il processo di integrazione e a ridurre duplicazioni e incoerenze nel codice.

Fase di Inception

La documentazione di inception, disponibile nell'allegato **Documentazione Generale**, comprende:

- Modellazione dei casi d'uso;
- Modello di dominio;
- Glossario dei termini principali.

Questa fase ha permesso di chiarire il perimetro funzionale del sistema e ha portato all'identificazione dei seguenti **Use Case**:

- **UC1** – Registrazione
- **UC2** – Login
- **UC3** – Visualizza note
- **UC4** – Crea nota
- **UC5** – Visualizza versioni
- **UC6** – Duplica nota
- **UC7** – Rimuove il proprio utente da nota condivisa
- **UC8** – Cerca una nota
- **UC9** – Aggiunge a cartella
- **UC10** – Modifica nota
- **UC11** – Modifica permessi
- **UC12** – Elimina nota

Organizzazione del Gruppo e Regole di Collaborazione

Nel corso dello Sprint 0, il gruppo ha definito un insieme di regole condivise per garantire ordine, efficienza e trasparenza nel lavoro di squadra.

- **Pull Request**
 - creata sempre dal **Developer 1**;
 - approvata esclusivamente dallo **Scrum Master**, in quanto terza parte imparziale.
- **Pair Programming**
 - Developer 1 e Developer 2 lavorano sempre insieme;
 - rotazione ordinata dei ruoli ad ogni sprint.

La rotazione lineare dei ruoli è stata definita anche per favorire **continuità e trasferimento di competenze operative**: in particolare, la presenza nello sprint successivo di un membro che aveva già gestito le attività di integrazione nello sprint precedente ha facilitato la trasmissione di procedure (branching, Pull Request, esecuzione test, verifica finale), riducendo l'onboarding interno e rendendo più uniforme l'applicazione del workflow.

Stima del tempo e capacità del team

In fase di pianificazione, il gruppo ha stimato una **capacità media di circa 24 ore per sprint** (team capacity), usata come riferimento per bilanciare backlog e obiettivi settimanali. Questa scelta ha aiutato a:

- mantenere obiettivi realistici e sostenibili;
- distribuire meglio sviluppo, integrazione e stabilizzazione;
- rendere più coerente la retrospettiva tra lavoro previsto e lavoro effettivamente svolto.

Comunicazione

- utilizzo di **WhatsApp e Microsoft Teams** per allineamenti quotidiani e confronto tecnico;
- utilizzo di una **Agenda/To-Do List condivisa** per tracciare proposte, attività completate e aspetti migliorativi.

Retrospettiva dello Sprint 0

Nonostante non fossero previste implementazioni tecniche, lo Sprint 0 ha già evidenziato alcuni punti di attenzione, in particolare la necessità di:

- uniformare lo stile di programmazione fin dalle prime fasi;
- chiarire il funzionamento dei branch Git;
- definire procedure di review chiare e non ambigue.

La retrospettiva ha confermato che il team ha costruito una base solida, organizzata e ben documentata, pronta a sostenere lo Sprint 1.

Conclusione

Lo Sprint 0 ha raggiunto pienamente tutti gli obiettivi previsti: definizione dello stack tecnologico, organizzazione del gruppo di lavoro, creazione e configurazione della repository GitHub, modellazione del dominio applicativo e dei casi d'uso, e impostazione delle regole di collaborazione.

Questa fase preparatoria ha costituito un riferimento essenziale per l'intero progetto, consentendo di avviare le iterazioni successive su basi chiare, condivise e strutturate.

Sprint 1 — Implementazione da UC1 a UC4 e MVP iniziale

Periodo: 03/11/2025 – 09/11/2025

Lo Sprint 1 ha segnato l'avvio operativo dello sviluppo dell'applicazione *Nota Bene*, rappresentando il primo sprint in cui il team ha prodotto un incremento software concreto. In questa iterazione è stato realizzato il **primo Minimum Viable Product (MVP)** del sistema, costruendo le fondamenta tecniche e funzionali dell'applicazione.

L'obiettivo dello sprint non era introdurre da subito un sistema completo e persistente, bensì costruire una prima base eseguibile e verificabile che permettesse di: (i) validare il flusso utente principale, (ii) definire l'architettura iniziale (modelli, servizi, controller), (iii) stabilire le prime API REST, e (iv) predisporre un set minimo di test automatici. In particolare, la persistenza è stata **deliberatamente limitata alla memoria (in-memory)** tramite strutture dati (es. HashMap), rinviando l'introduzione di un repository persistente agli sprint successivi, in modo da ridurre complessità e rischi nelle fasi iniziali.

Ruoli assegnati

In continuità con il modello organizzativo stabilito nello Sprint 0, i ruoli per lo Sprint 1 sono stati assegnati come segue:

- **Product Owner (PO):** Lorenzo
- **Scrum Master (SM):** Alessia — responsabile approvazione delle Pull Request
- **Developer 1:** Antonella — responsabile dell'apertura delle Pull Request
- **Developer 2:** Aurora

Il workflow definito nello Sprint 0 (Pull Request create da Developer 1 e approvate esclusivamente dallo Scrum Master, pair programming costante e rotazione dei ruoli) è stato applicato in modo rigoroso e coerente fin dal primo sprint di sviluppo.

Sprint Planning

Fase 1 — Selezione dei Casi d'Uso

Il Product Owner ha selezionato per lo Sprint 1 i seguenti **Use Case**, ritenuti essenziali per garantire una prima versione minima ma utilizzabile del sistema:

- UC1 – Registrazione
- UC2 – Login
- UC3 – Visualizza note (limitato alle note personali)
- UC4 – Crea nota

La priorità attribuita a questi casi d'uso è derivata dalla necessità di disporre fin da subito di:

- un sistema utenti funzionante e verificabile;
- una prima dashboard capace di mostrare contenuti associati all'utente autenticato;
- un insieme minimo di endpoint REST su cui basare l'evoluzione successiva;
- una prima gestione delle note in modalità **non persistente** (in-memory), sufficiente per test funzionali e validazione del flusso principale.

Il lavoro previsto per lo sprint comprendeva, quindi:

- implementazione del backend per utenti e note (modelli, repository in-memory, service, controller);
- definizione delle API REST minime;
- realizzazione di un frontend essenziale a supporto delle verifiche;
- predisposizione di una prima suite di test automatici focalizzata sulle funzionalità implementate.

Fase 2 — Discussione e suddivisione dei task

Le attività sono state distribuite in modo chiaro tra i membri del team.

Product Owner (Lorenzo)

- Validazione dei requisiti implementati;
- Testing dei casi limite;
- Verifica complessiva delle funzionalità prima della Sprint Review.

Scrum Master (Alessia)

- Supervisione del repository;
- Revisione e approvazione delle Pull Request;
- Gestione della board GitHub e coordinamento giornaliero.

Developer 1 – Backend (Antonella)

- Implementazione del sistema utenti (UserService, UserRepository in-memory, UserController);
- Definizione degli endpoint di registrazione e login;
- Implementazione del primo nucleo di gestione note (NoteService, NoteRepository in-memory, NoteController);
- Modellazione iniziale delle entità Utente e Nota;
- Aggiornamento configurazione Maven (pom.xml) e .gitignore;
- Creazione della prima suite di test automatici con JUnit;
- Refactoring iniziale per uniformare struttura e naming.

Developer 2 – Frontend e Support (Aurora)

- realizzazione dell'interfaccia per registrazione e login;
- pagine e script JavaScript per creazione e visualizzazione note;
- integrazione frontend-backend tramite chiamate fetch() alle API REST.

Sprint Backlog – Sprint 1

UC Raffinati	Product Backlog	Sprint Backlog	Task Terminati
UC1 – Registrazione UC2 – Login UC3 – Visualizza note UC4 – Crea nota UC5 – Visualizza versioni UC6 – Duplica nota UC7 – Rimuove utente UC8 – Cerca una nota UC9 – Aggiunge a cartella UC10 – Modifica nota UC11 – Modifica permessi UC12 – Elimina nota	UC5 UC6 UC7 UC8 UC9 UC11 UC12	UC1 UC2 UC3 - Note personali UC4	UC1 – Implementato UC2 – Implementato UC3 – Implementato (versione personale delle note) UC4 – Implementato (creazione note)

Sprint Review

Durante la Sprint Review, il team ha presentato un incremento pienamente funzionante, comprendente:

➤ UC1 – Registrazione

- Implementazione backend funzionante;
- Validazione essenziale dei campi;
- Repository utenti in-memory;
- Interfaccia frontend semplice con messaggi di errore.

➤ UC2 – Login

- Verifica corretta delle credenziali;
- Instradamento dell'utente verso la dashboard in caso di autenticazione riuscita.

➤ UC3 – Visualizzazione Note personali

- Recupero delle note associate all'utente loggato tramite API REST;
- Rendering funzionale lato client.

➤ UC4 – Creazione Nota

- API REST /api/notes correttamente implementata;
- Salvataggio delle note in memoria (non persistente, in questa fase);
- Interfaccia con campi titolo e contenuto.

Il Product Owner ha testato l'intero flusso:

Registrazione → Login → Creazione Nota → Visualizzazione Nota
confermando che tutte le funzionalità risultavano stabili e conformi allo scope dello sprint.

Il commit finale dello sprint riassume l'incremento consegnato:

"Sprint 1: completati UC1-UC4. Aggiunta suite di test, pulizia backend, frontend minimale e aggiornamento pom.xml e .gitignore."

Sprint Retrospective

Durante la retrospettiva sono emersi alcuni aspetti da migliorare:

Problemi riscontrati

- Iniziali incertezze nella gestione delle branch Git;
- Divergenze nello stile di programmazione tra i developer;
- Necessità di un metodo di coordinamento più strutturato.

Azioni di miglioramento introdotte

- Conferma di WhatsApp come canale di comunicazione principale;
- Pair programming costante tra Developer 1 e Developer 2;
- Introduzione di un'agenda / to-do list condivisa per annotare idee, segnalare bug e pianificare miglioramenti.

Regole consolidate

- Le Pull Request vengono create da Developer 1;
- Le Pull Request vengono approvate solo dallo Scrum Master;
- Pair programming obbligatorio;
- Rotazione dei ruoli ad ogni sprint;
- Agenda condivisa come punto di riferimento organizzativo del team.

Conclusione

Lo Sprint 1 ha raggiunto pienamente gli obiettivi prefissati, consentendo l'implementazione dei primi quattro casi d'uso e la realizzazione dell'MVP dell'applicazione *Nota Bene*.

In questa fase è stato definito il nucleo minimo dell'architettura (modelli, servizi, controller, API REST) e un frontend essenziale per la validazione del flusso principale. La persistenza è stata mantenuta volutamente **in-memory**, rinviando la gestione persistente dei dati agli sprint successivi, in modo da procedere per incrementi controllati e progressivi.

Sprint 2 — Estensione UC3–UC4 e Implementazione UC6, UC8, UC9, UC12

Periodo: 17/11/2025 – 23/11/2025

Lo **Sprint 2** ha avuto come obiettivo l'estensione controllata dell'MVP realizzato nello Sprint 1, introducendo un primo insieme di funzionalità tipiche di un sistema di note-taking, mantenendo tuttavia un approccio incrementale orientato al **minimo** per ciascun caso d'uso selezionato.

In questa iterazione sono state introdotte operazioni essenziali per la gestione delle note: **uplicazione, ricerca, organizzazione in cartelle** (in forma iniziale) ed **eliminazione**. Inoltre, UC4 è stato esteso rispetto allo sprint precedente, al fine di migliorare la fruibilità della creazione nota e predisporre la dashboard a una gestione più articolata delle funzionalità.

È importante evidenziare che, in questo sprint, **UC3 è rimasto limitato alla visualizzazione delle note personali**, senza includere ancora la visualizzazione delle note condivise. Tale scelta è stata deliberata: la componente relativa alle note condivise richiedeva un allineamento complessivo su regole e criteri (visibilità, permessi, casi limite) che il team ha pianificato di definire in modo congiunto durante un incontro in presenza. Di conseguenza, l'estensione di UC3 alle note condivise è stata rinviata agli sprint successivi per garantire coerenza progettuale e ridurre il rischio di rework.

Ruoli assegnati

In continuità con il modello di rotazione stabilito nello Sprint 0, i ruoli per questa iterazione sono stati assegnati come segue:

- **Product Owner (PO):** Aurora
- **Scrum Master (SM):** Lorenzo — responsabile approvazione delle Pull Request
- **Developer 1:** Alessia — responsabile dell'apertura delle Pull Request
- **Developer 2:** Antonella

La rotazione dei ruoli ha favorito una crescita equilibrata delle competenze tecniche e organizzative dei membri del team, consentendo a ciascun partecipante di confrontarsi con responsabilità differenti.

Sprint Planning

Fase 1 — Selezione dei Casi d'Uso

Il Product Owner ha selezionato i seguenti Use Case per lo Sprint 2, ritenuti prioritari per migliorare l'esperienza utente nella gestione delle note:

- UC4 – Crea nota, estensione e miglioramento rispetto allo Sprint 1
- UC6 – Duplica nota
- UC8 – Cerca una nota
- UC9 – Aggiunge a cartella
- UC12 – Elimina nota

UC3 — Visualizza note è stato mantenuto **limitato alle note personali**, rinviando volutamente la parte relativa alle note condivise agli sprint successivi (motivazione riportata nell'introduzione).

La scelta di questi casi d'uso è stata motivata dalla necessità di introdurre azioni operative sulla nota (creazione, duplicazione, eliminazione), migliorare la fruibilità tramite ricerca, e avviare un primo livello di organizzazione tramite cartelle, mantenendo un perimetro implementativo essenziale.

Fase 2 — Suddivisione dei Task

Le attività sono state suddivise in modo chiaro tra i membri del team.

Product Owner (Aurora)

- Validazione incrementale delle funzionalità sviluppate;
- Test funzionali e controllo dei casi limite;
- Revisione delle priorità per lo Sprint 3.

Scrum Master (Lorenzo)

- Revisione del codice prodotto;
- Gestione del flusso delle Pull Request;
- Aggiornamento della board Kanban;
- Coordinamento tecnico durante la fase di integrazione.

Developer 1 – Backend e Pull Request (Alessia)

- aggiornamento NoteService e NoteRepository per: duplicazione (UC6), assegnazione a cartella (UC9), eliminazione (UC12);
- estensione delle API REST per i nuovi casi d'uso;
- introduzione/estensione test JUnit sulle funzionalità aggiunte.

Developer 2 – Frontend e Integrazione Backend (Antonella)

- aggiornamento dashboard per: pulsante “duplica” (UC6), pulsante “elimina” (UC12), selettore cartelle (UC9);
- implementazione della barra di ricerca con filtraggio lato client (UC8);
- integrazione funzionalità frontend ↔ API REST mediante fetch().

Sprint Backlog – Sprint 2

UC Raffinati	Product Backlog	Sprint Backlog	Task Terminati
UC1 – Registrazione UC2 – Login UC3 – Visualizza note UC4 – Crea nota UC5 – Visualizza versioni UC6 – Duplica nota UC7 – Rimuove utente UC8 – Cerca una nota UC9 – Aggiunge a cartella UC10 – Modifica nota UC11 – Modifica permessi UC12 – Elimina nota	UC5 UC7 UC10 UC11	UC3 – Visualizza note (include note condivise) UC4 – Crea nota (estesa) UC6 – Duplica nota UC8 – Cerca nota UC9 – Aggiunge a cartella UC12 – Elimina nota	UC3 – Parsing note condivise UC4 – Miglioramento ed estensione creazione nota UC6 – Duplica nota UC8 – Ricerca UC9 – Cartelle UC12 – Eliminazione nota

Output consegnati

- API REST aggiornate per duplicazione, ricerca, eliminazione e gestione cartelle;
- Interfaccia dashboard estesa con nuovi pulsanti e sezioni;
- Vista dell note condivise pienamente funzionante;
- Ricerca in tempo reale lato frontend;
- Gestione delle cartelle integrata a livello base;
- Eliminazione della nota con aggiornamento dinamico della lista.

Sprint Review

Il team ha completato tutte le funzionalità previste, ottenendo un sistema più completo e vicino al comportamento desiderato per l'applicazione.

➤ UC4 – Creazione nota (estesa)

- Miglioramento dell'interfaccia utente;
- Primo supporto alle cartelle in fase di creazione.

➤ UC6 – Duplicazione nota

- Duplicazione completa lato backend con generazione di un nuovo ID univoco;
- Pulsante dedicato con feedback immediato lato frontend.

➤ UC8 – Cercare una nota

- Ricerca reattiva direttamente sulla dashboard;
- Possibilità di filtrare note personali e note condivise.

➤ UC9 – Aggiungere a cartella

- Menù dedicato per l'assegnazione della cartella;
- Aggiornamento immediato della visualizzazione.

➤ **UC12 – Eliminazione nota**

- Eliminazione lato backend con controlli sui casi limite;
- Refresh automatico della UI;
- Controllo errori e casi limite.

In sede di Sprint Review, il team ha confermato che la visualizzazione delle note condivise non rientrava nello scope implementativo dello Sprint 2. La funzionalità è stata intenzionalmente posticipata, in quanto dipendente da decisioni condivise su permessi e regole di collaborazione; tali decisioni sono state pianificate per una sessione di lavoro in presenza con tutti i membri del gruppo, così da definire un comportamento univoco e ridurre successive modifiche correttive.

Durante la demo di Sprint Review, il Product Owner ha verificato con successo l'intero flusso:

Creazione → Duplicazione → Spostamento → Eliminazione

oltre alla ricerca delle note e al passaggio tra note personali e condivise.

Tutte le funzionalità hanno mostrato stabilità e coerenza con lo scope dello sprint.

Il commit finale dello sprint riassume l'incremento consegnato:

“Sprint 2: implementazione UC3, UC4, UC6, UC8, UC9 e UC12.”

Sprint Retrospective

Durante la retrospettiva sono emersi alcuni aspetti da migliorare:

Problemi riscontrati

- Necessità di uniformare ulteriormente la gestione delle cartelle
- Aumento della complessità della dashboard
- Alcune difformità stilistiche tra componenti JavaScript

Azioni di miglioramento

- Refactoring previsto per lo Sprint 3
- Rafforzamento del controllo qualità nelle Pull Request
- Razionalizzazione del naming e dei commenti nel frontend

Regole confermate

- Pull Request approvate solo dallo Scrum Master
- Developer 1 crea la Pull Request; Developer 2 collabora in pair programming
- Rotazione dei ruoli mantenuta
- Agenda condivisa utilizzata quotidianamente per comunicazioni, note e attività

Conclusione

Lo Sprint 2 ha ampliato l'MVP introducendo funzionalità operative centrali (duplicazione, ricerca, cartelle in forma iniziale, eliminazione) e migliorando la creazione nota (UC4). L'incremento prodotto costituisce una base più ricca e utilizzabile, preparatoria agli aspetti più complessi previsti dal progetto (note condivise, permessi, versionamento e collaborazione), affrontati nelle iterazioni successive.

Sprint 3 — Implementazione UC5, UC7, UC10, UC11, persistenza MapDB e frontend avanzato

Periodo: 24/11/2025 – 30/11/2025

Lo **Sprint 3** ha rappresentato una fase di consolidamento e ristrutturazione significativa per *Nota Bene*. Dopo due sprint orientati alla costruzione dell'MVP e all'introduzione delle funzionalità operative di base, questa iterazione si è concentrata su due obiettivi principali:

1. **stabilizzare l'architettura** introducendo la persistenza tramite **MapDB** (superando il repository in-memory);
2. **definire in modo condiviso** le modalità di interazione tra le componenti più complesse del dominio (condivisioni, permessi, versioni e flussi di modifica), grazie anche a **tre giornate di lavoro in presenza**.

Considerata la maggiore complessità tecnica e organizzativa dello sprint, il team ha pianificato una capacità complessiva pari a **36 ore/settimana** (stima riferita all'intero gruppo). Tale incremento rispetto agli sprint precedenti è stato motivato dall'introduzione della persistenza tramite MapDB, dalla necessità di definire e validare regole di interazione più articolate (condivisioni, permessi, versionamento) e dalla presenza di **tre giornate di lavoro in presenza**, che hanno favorito sessioni di sviluppo e integrazione più continue e una Pull Request finale più estesa.

Le sessioni in presenza hanno avuto un impatto rilevante sulla qualità delle decisioni progettuali: hanno consentito di allineare l'intero gruppo su regole, casi limite e comportamenti attesi, riducendo il rischio di incoerenze tra backend e frontend e limitando rework nelle fasi di integrazione. Parallelamente, la dashboard è stata evoluta verso una versione più completa e curata, con un'interazione utente più chiara e un set più ampio di azioni contestuali.

Ruoli assegnati

In conformità con la rotazione prevista dagli sprint precedenti, lo Sprint 3 ha visto la seguente assegnazione dei ruoli:

- **Product Owner (PO): Antonella**
- **Scrum Master (SM): Aurora** — responsabile approvazione delle Pull Request
- **Developer 1:** Lorenzo — responsabile dell'apertura delle Pull Request
- **Developer 2:** Alessia

La distribuzione dei ruoli ha favorito un equilibrio efficace tra progettazione funzionale, sviluppo backend, UI avanzata e revisione del codice.

Sprint Planning

Fase 1 - Selezione dei Casi d'Uso

Il Product Owner ha selezionato per lo Sprint 3 gli Use Case che, per complessità e dipendenze, richiedevano un consolidamento architetturale (persistenza) e un allineamento condiviso sulle regole di dominio. In particolare, lo sprint ha incluso:

- UC5 – Visualizza versioni
- UC7 – Rimuove utente da nota condivisa
- UC10 – Modifica nota
- UC11 – Modifica permessi

oltre al completamento/rafforzamento in chiave persistente e “avanzata” di funzionalità già introdotte negli sprint precedenti (UC3, UC4, UC6, UC8, UC9, UC12), in modo coerente con MapDB e con le nuove regole concordate.

Fase 2 — Suddivisione dei Task

Product Owner (Antonella)

- Test completi delle funzionalità avanzate: crea, visualizza, duplica, cancella, ricerca
- Validazione delle regole di business applicate: permessi, versioni, condivisioni e cartelle
- Revisione dei flussi per assicurare consistenza in vista dello Sprint 4

Scrum Master (Aurora)

- Revisione del codice backend e frontend
- Merge delle Pull Request
- Controllo coerenza tra funzionalità backend e interfacce frontend
- Aggiornamento board Kanban e coordinamento quotidiano

Developer 1 – Backend MapDB e API definitive (Lorenzo)

- Introduzione e configurazione completa di MapDB come sistema di persistenza stabile
- Refactoring completo di NoteRepository con mappe persistenti per note, versioni, cartelle e condivisioni
- Implementazione CRUD definitiva: create, update, delete, duplicate, restore
- Gestione cartelle: creazione, assegnazione, pulizia cartelle vuote
- Aggiornamento di tutti gli endpoint REST (NoteController)
- Implementazione della ricerca lato backend con filtri completi
- Definizione delle logiche di condivisione e permessi

Developer 2 – Frontend avanzato (Alessia)

- Evoluzione della dashboard con rendering dinamico e azioni contestuali (modifica, duplica, elimina, sposta, restore dove previsto);
- Revisione delle modali (crea/modifica) per renderle coerenti e riutilizzabili;
- Ricerca e filtri lato UI;
- Gestione degli aggiornamenti del DOM dopo operazioni CRUD;
- Introduzione/miglioramento feedback utente (toast, aggiornamenti immediati, evidenziazioni).

Sprint Backlog – Sprint 3

UC Raffinati	Product Backlog	Sprint Backlog	Task Terminati
UC1 – Registrazione UC2 – Login UC3 – Visualizza note UC4 – Crea nota UC5 – Visualizza versioni UC6 – Duplica nota UC7 – Rimuove utente UC8 – Cerca una nota UC9 – Aggiunge a cartella UC10 – Modifica nota UC11 – Modifica permessi UC12 – Elimina nota	UC5 UC7 UC10 UC11	UC3 UC4 UC5 UC6 UC7 UC8 UC9 UC10 UC11 UC12	UC3 – Implementato UC4 – Implementato UC5 – Implementato (versioni + restore) UC6 – Implementato UC7 – Implementato UC8 – Implementato UC9 – Implementato UC10 – Implementato (modifica nota) UC11 – Implementato (permessi) UC12 – Implementato

Gli UC completati nello Sprint 3, nella loro versione “avanzata e definitiva”, sono:

- **UC3 – Visualizza note** (versione avanzata: note condivise, filtri, MapDB)
- **UC4 – Crea nota** (versione definitiva: persistenza + cartelle)
- **UC6 – Duplica nota** (integrazione definitiva con MapDB)
- **UC8 – Cerca una nota** (backend + frontend avanzato)
- **UC9 – Aggiunge a cartella** (con persistenza effettiva)
- **UC12 – Elimina nota** (con aggiornamento dinamico UI + DB)

Output consegnati

Backend (architettura e funzionalità)

- Introduzione della persistenza tramite MapDB, con salvataggio dei dati e mantenimento tra riavvii dell'applicazione.
- Refactoring dei repository e consolidamento del service layer per supportare operazioni complete sulle note.
- Aggiornamento e stabilizzazione delle API REST per le operazioni principali (creazione, lettura, aggiornamento, eliminazione, duplicazione e ricerca).
- Implementazione/integrazione delle regole di business relative a:
 - condivisione note e gestione utenti coinvolti; permessi (lettura/scrittura/privata secondo regole definite nello sprint);
 - versionamento con possibilità di consultazione e ripristino (restore) dove previsto.
- Gestione della ricerca lato backend (filtri e recupero note coerente con la visibilità dell'utente).

Frontend (dashboard avanzata)

- Evoluzione della dashboard verso una versione più articolata, con:
 - rendering dinamico delle note; azioni contestuali (apri/modifica, duplica, elimina, ripristino versione dove previsto);
 - aggiornamenti immediati della UI dopo operazioni CRUD (refresh parziale della lista e gestione stati).
- Introduzione/miglioramento del sistema di modali per creazione e modifica nota, più coerenti e riutilizzabili.
- Implementazione della ricerca lato UI (filtri e aggiornamento dinamico della lista note).
- Introduzione di feedback utente più strutturati (es. toast e messaggi di conferma/errore) per rendere più chiari gli esiti delle azioni.

Integrazione e qualità

- Allineamento progressivo tra comportamento atteso e comunicazione frontend-backend (parametri, formati dati, gestione errori).
- Test end-to-end sui flussi principali per verificare consistenza e stabilità dell'incremento.

Cartelle e Lock System (versione preliminare)

Nel corso dello Sprint 3 sono stati introdotti anche i primi elementi dei sistemi di **cartelle** e **locking** in una forma preliminare, con l'obiettivo di preparare l'integrazione completa negli sprint successivi:

- **Cartelle:** è stata implementata principalmente la componente **backend/persistenza** (gestione e assegnazione della cartella alle note e salvataggio su MapDB), mentre la parte frontend risultava ancora parziale e non pienamente operativa.
- **Locking:** è stata introdotta la **logica di base** (struttura e primi controlli), ma il meccanismo non garantiva ancora il **blocco effettivo** della nota lato interfaccia e la gestione completa dei casi concorrenti; tali aspetti sono stati consolidati nello Sprint 4.

Sprint Review

Durante la review, il team ha verificato che tutte le nuove funzionalità fossero pienamente operative:

- **Backend MapDB completato**
 - Dati persistenti tra riavvii
 - Versionamento e restore corretti
 - Permessi e condivisioni funzionali
 - Gestione cartelle come entità separate
 - CRUD stabile e completo
 - Ricerca avanzata lato backend
- **Dashboard Frontend avanzata**
 - Render dinamico delle note
 - Azioni contestuali complete (modifica, duplica, elimina, restore, sposta)
 - Ricerca in tempo reale
 - Modali aggiornate
 - Visualizzazione note condivise con filtri
 - UI sincronizzata con il backend persistente
- **Flussi testati con successo**
 - Crea → Modifica → Duplica → Elimina → Cerca → Restore
 - Spostamento note in cartelle
 - Interazione con note condivise
 - Persistenza dati verificata al riavvio dell'applicazione

Lo Sprint 3 ha mostrato un sistema solido, coerente e perfettamente funzionante.

Sprint Retrospective

Nel corso dello Sprint 3 è emersa con chiarezza la necessità di prestare maggiore attenzione alla manutenibilità del codice JavaScript: l'aumento del numero di funzionalità e di stati UI ha reso la dashboard progressivamente più complessa, incrementando il rischio di regressioni e di difficoltà nella gestione del DOM. In prospettiva, sarà opportuno pianificare un ulteriore lavoro di refactoring e razionalizzazione del codice frontend (funzioni più modulari, separazione responsabilità, naming uniforme), così da mantenere controllabile l'evoluzione dell'interfaccia.

Lo sprint è risultato particolarmente intenso anche dal punto di vista dell'integrazione: la definizione delle regole di interazione (condivisioni, permessi, versionamento) ha richiesto un confronto continuo e, in più momenti, sono state riscontrate difficoltà iniziali nella corrispondenza tra API backend e azioni frontend (parametri attesi, gestione errori, aggiornamento dinamico della UI). Tali criticità sono state risolte progressivamente attraverso test end-to-end e iterazioni di allineamento.

Infine, nella fase di integrazione è stato riscontrata una criticità legata alla Pull Request, che ha richiesto supporto aggiuntivo nella gestione del flusso di revisione e merge; in tale contesto, Antonella ha svolto un ruolo di supporto per favorire la risoluzione tempestiva e mantenere lo sprint entro il time-box.

Punti critici individuati

- Aumento della complessità del frontend (JavaScript), con crescita del rischio di regressioni e minore manutenibilità della dashboard.
- Gestione del DOM sempre più delicata, a causa del numero crescente di stati UI e aggiornamenti dinamici dopo operazioni CRUD.
- Allineamento backend–frontend non immediato: in alcuni casi è stato necessario iterare per garantire corrispondenza tra endpoint, parametri attesi, gestione errori e azioni UI.
- Criticità nella fase di integrazione/Pull Request, che ha richiesto supporto aggiuntivo per la risoluzione e il merge (con Antonella in ruolo di supporto).

Azioni migliorative

- Pianificazione di un refactoring del frontend (JS) nello Sprint 4: maggiore modularità, separazione delle responsabilità e naming coerente.
- Consolidamento della gestione UI (DOM + stati) tramite standardizzazione di modali, pulsanti e feedback e riduzione di codice duplicato.
- Miglioramento della documentazione interna sulle scelte MapDB e sulle regole di interazione (permessi/condivisioni/versioni), per ridurre ambiguità in integrazione.
- Rafforzamento del controllo in fase di review: checklist di coerenza API ↔ UI prima del merge.

Regole confermate

- Pull Request approvate esclusivamente dallo Scrum Master.
- Rotazione dei ruoli mantenuta.
- Pair programming tra Developer 1 e Developer 2.
- Agenda condivisa e comunicazione quotidiana (WhatsApp/Teams) per coordinamento e feedback.

Conclusione

Lo Sprint 3 ha segnato il passaggio da un MVP basato su gestione semplice dei dati a un sistema più strutturato e coerente, grazie all'introduzione della persistenza MapDB, alla definizione condivisa delle regole di interazione tra le componenti più complesse (condivisioni, permessi, versioni), e a un'evoluzione significativa del frontend. Queste basi rendono possibile dedicare lo Sprint 4 a rifiniture, consolidamento, qualità dell'esperienza utente e ulteriore standardizzazione dell'interfaccia.

Sprint 4 — Consolidamento Lock System, Cartelle (frontend), DTO e test manuali strutturati

Periodo: 01/12/2025 – 07/12/2025

Lo Sprint 4 ha rappresentato una fase di consolidamento fondamentale per il progetto *Nota Bene*. Dopo l'introduzione e la strutturazione delle funzionalità principali nello Sprint 3 (persistenza MapDB, versionamento, permessi e condivisioni, prime logiche per cartelle e locking), questa iterazione ha avuto l'obiettivo di **completare l'integrazione** e rendere i meccanismi pienamente operativi sul frontend, migliorando in modo significativo l'esperienza utente e la coerenza tra API e interfaccia. In continuità con lo Sprint 3, il team ha pianificato una **capacity complessiva pari a 36 ore/settimana** (stima riferita all'intero gruppo).

Il lavoro si è concentrato su cinque aree chiave:

- **Lock System** completato lato UI e consolidato lato backend, con gestione della concorrenza, timer e popup di avviso.
- **Modularizzazione del frontend** tramite introduzione di un file dedicato (lock-system.js) richiamato da dashboard.js, con l'obiettivo di ridurre la complessità e migliorare la manutenibilità.
- **Versionamento** con interfaccia migliorata e flusso completo di restore.
- **Cartelle** completate lato frontend (icone, colori, selettori e aggiornamenti dinamici).
- **Comunicazione API** stabilizzata tramite introduzione/standardizzazione di DTO, rendendo i contratti dati più espliciti e riducendo errori di integrazione.

In parallelo, lo Sprint 4 ha introdotto una fase di **test manuali più strutturati**, con tracciamento dei bug tramite codifica per autore (es. L1/L2/..., A1/A2/...), migliorando la comunicazione interna e la prioritizzazione delle correzioni.

Ruoli assegnati:

Come da rotazione programmata, i ruoli nello Sprint 4 sono stati:

- **Product Owner (PO):** Alessia
- **Scrum Master (SM):** Antonella — responsabile approvazione delle Pull Request
- **Developer 1:** Aurora — responsabile dell'apertura delle Pull Request
- **Developer 2:** Lorenzo

Questa configurazione ha permesso un equilibrio tra controllo qualità, sviluppo frontend avanzato e supporto backend.

Sprint Planning

Fase 1 — Selezione dei Casi d'Uso

Nello Sprint 4 non sono stati introdotti nuovi Use Case.

L'obiettivo era il perfezionamento degli UC già presenti, in particolare:

- UC5 – Visualizza versioni (raffinamento UI + restore)
- UC7 – Rimuove utente da nota condivisa (UI polishing)
- UC8 – Cerca una nota (UI + filtri)
- UC9 – Aggiunge a cartelle (colori, icone, selezione)
- UC10 – Modifica nota (flusso con lock system)
- UC12 – Elimina nota (aggiornamento dinamico)

e l'integrazione completa dei sistemi trasversali:

- Lock System
- Dashboard frontend avanzata

Fase 2 — Suddivisione dei Task

Le attività sono state distribuite in modo chiaro tra i membri del team:

Product Owner (Alessia)

- Validazione completa dei flussi:
 - modifica nota con lock
timeout lock + annulla/modifica
 - restore versione
 - cambio cartella
- Test end-to-end della dashboard
- Segnalazione bug e feedback su UI/UX

Scrum Master (Antonella)

- Controllo qualità Pull Request
- Verifica coerenza API ↔ frontend
- Coordinamento dei test incrementali
- Aggiornamento board GitHub

Developer 1 – Frontend avanzato + PR (Aurora)

- Implementazione completa della UI del lock system (modali di avviso, timer secondi rimanenti, popup conferma salvataggio)
- Aggiornamento dashboard:
 - refresh dinamico dopo restore e dopo lock scaduto
 - pulsanti uniformati e icone aggiornate
 - toast coerenti di successo/errore e conferme per tutte le funzioni
- Riorganizzazione della modale “Modifica nota” per integrare versioni e cartelle
- Apertura Pull Request

Developer 2 – Backend + integrazione frontend (Lorenzo)

- Revisione e completamento API lock/unlock
- Integrazione backend-frontend per il versionamento:
 - passaggio corretto delle versioni alla UI
 - gestione index versione e ripristino
 - restore e aggiornamento badge versione nella dashboard
- Miglioramento logica cartelle nel backend e sincronizzazione frontend
- Supporto al debugging del caricamento note nella dashboard

Sprint Backlog – Sprint 4

UC Raffinati	Product Backlog	Sprint Backlog	Task Terminati
UC1 – Registrazione UC2 – Login UC3 – Visualizza note UC4 – Crea nota UC5 – Visualizza versioni UC6 – Duplica nota UC7 – Rimuove utente UC8 – Cerca una nota UC9 – Aggiunge a cartella UC10 – Modifica nota UC11 – Modifica permessi UC12 – Elimina nota	Nessun nuovo UC: solo perfezionamenti e integrazioni frontend-backend	UC5 – Versioni (UI) UC7 – Rimuovi se stesso (UI) UC8 – Ricerca (UI + filtri) UC9 – Cartelle (UI definitiva) Lock System – integrazione UX Dashboard – aggiornamenti completi	Lock system frontend Popup, timer, conferme Restore versioni – UI completa Cartelle – icone + colori + selettore Dashboard redesign Toast coerenti Fix DOM e refresh note

Durante lo Sprint 4, sono stati completati i seguenti ambiti:

Lock System (completo e integrato)

- Timer di modifica attiva e prevenzione accessi concorrenti.
- Popup di avviso e conferma (“Stai ancora modificando?” dove previsto).
- Sblocco lato backend in caso di timeout e aggiornamento UI coerente.

Refactoring frontend: modularizzazione del locking

- Separazione della logica in un file dedicato (**lock-system.js**) richiamato da **dashboard.js**, riducendo l'accoppiamento e migliorando la leggibilità del codice.

Versionamento (raffinato)

- Modale versioni integrata e usabile.
- Restore con aggiornamento immediato del DOM e del badge versione.
- Correzione bug relativi a ordine versioni e refresh parziale.

Cartelle (frontend completato)

- Icone e colori dinamici persistenti.
Selettore cartelle nelle modali nota e miglioramento della pagina cartelle.html.
- Aggiornamento note in base alla cartella selezionata.

DTO e stabilizzazione comunicazione backend–frontend

- Introduzione/standardizzazione dei **DTO** per rendere espliciti i dati necessari alle azioni UI e ridurre mismatch tra strutture dati e richieste del frontend.

Test manuali strutturati

- Avvio di test end-to-end più rigorosi.
- Tracciamento bug tramite codifica per autore (L1/L2/...; A1/A2/...), utilizzata per facilitare assegnazione, discussione e verifica delle correzioni.

Sprint Review

Durante la review, il team ha verificato in modo esteso l'integrazione completa tra frontend e backend per:

- apertura nota → lock attivo
- scadenza lock → popup → aggiornamento automatico
- restore versione → UI aggiornata immediatamente
- cambio cartella → dashboard aggiornata senza ricaricare la pagina
- eliminazione e duplicazione → sincronizzazione lista note
- gestione colori cartelle e icone pienamente funzionali

Tutti i flussi principali sono risultati coerenti e pienamente funzionanti.

Il PO ha confermato che:

- la UX è ora fluida e comprensibile
- il lock system non produce conflitti
- il versionamento è stabile
- il comportamento cartelle è corretto anche su MapDB

Sprint Retrospective

Problemi riscontrati

- Complessità crescente della dashboard → necessità di refactoring futuro
- Alcuni comportamenti del lock difficili da simulare nei test manuali
- Debug del restore versione più complesso del previsto
- Conflitti nella gestione delle modali in dashboard.js, con aperture/sovrapposizioni e stati UI non sempre coerenti durante i test end-to-end.

Azioni migliorative

- Pianificare una revisione completa di dashboard.js nello Sprint 5
- Introdurre log di debug per lock/versioning
- Suddividere le funzioni JS in moduli più chiari

Regole confermate

- Developer 1 apre le Pull Request
- Pull Request approvate solo dallo Scrum Master
- Pair programming costante
- Agenda condivisa aggiornata quotidianamente

Conclusione

Lo Sprint 4 ha consolidato in modo significativo *Nota Bene*, completando l'integrazione tra un backend ormai strutturato e un frontend più maturo e coerente. In particolare, l'implementazione definitiva del Lock System, il completamento del frontend delle cartelle e la stabilizzazione della comunicazione tramite DTO hanno migliorato l'affidabilità delle interazioni e ridotto le ambiguità tra API e interfaccia. L'introduzione di test manuali strutturati, con codifica dei bug per autore, ha inoltre rafforzato la capacità del team di individuare regressioni e prioritizzare le correzioni in modo ordinato.

Durante tali test sono emerse anche alcune criticità legate alla gestione della UI nella dashboard, in particolare conflitti nell'apertura e nella gestione delle modali (sovrapposizioni o stati non sempre coerenti). Il team prevede di intervenire nello Sprint 5 per minimizzare tali conflitti e migliorare ulteriormente la stabilità dell'interfaccia. Tuttavia, l'elevata complessità raggiunta da dashboard.js rende il refactoring una fase delicata e impegnativa: rappresenta una sfida tecnica che richiederà interventi mirati, modularizzazione progressiva e un controllo attento delle regressioni.

Lo Sprint 5 potrà quindi concentrarsi su ottimizzazione, refactoring e polishing finale, con particolare attenzione a manutenibilità del frontend e robustezza dei flussi UI.

Sprint 5 — Bug Fixing, Stabilizzazione Lock-System, NoteView completa, Logging e Pulizia Codice

Periodo: 08/12/2025 – 14/12/2025

Lo **Sprint 5** ha rappresentato una fase di stabilizzazione cruciale per il progetto **Nota Bene**, con l'obiettivo di portare il sistema da una versione funzionalmente completa a un prodotto **coerente, affidabile e pronto per la consegna**. Dopo l'evoluzione architetturale e UI degli sprint precedenti, questa iterazione si è concentrata principalmente su: correzione dei bug emersi durante l'integrazione e durante la review delle specifiche, riallineamento di alcuni comportamenti alle richieste del documento di progetto (in particolare su **permessi e dati esposti dalla nota**), consolidamento della comunicazione frontend-backend tramite aggiornamenti ai **DTO**, uniformazione finale dell'interfaccia, oltre all'avvio e alla revisione della documentazione finale.

In continuità con gli Sprint 3 e 4 e considerata la natura di consolidamento dello sprint e l'effort richiesto dal lavoro di refactoring e stabilizzazione, il team ha pianificato una **capacity complessiva pari a 36 ore/settimana** (stima riferita all'intero gruppo), includendo attività di debugging, test manuali, fix di integrazione e **inizio della stesura/review della documentazione**.

Operativamente, lo sprint è stato articolato in **due Pull Request principali**, e di una **terza PR finale** orientata alla consegna di un database MapDB pulito e coerente (eventualmente con un set minimo di utenti già presenti per facilitare la demo e i test).

Ruoli assegnati

In conformità con la rotazione prevista dagli sprint precedenti, lo Sprint 5 ha visto la seguente assegnazione dei ruoli:

- **Product Owner (PO):** Lorenzo
- **Scrum Master (SM):** Alessia — responsabile approvazione delle Pull Request
- **Developer 1:** Antonella — responsabile dell'apertura delle Pull Request
- **Developer 2:** Aurora

L'equilibrio tra revisione backend, debugging frontend e validazione funzionale ha caratterizzato il lavoro di questo sprint.

Sprint Planning

Fase 1 — Selezione dei Casi d'Uso

Nello Sprint 5 **non sono stati introdotti nuovi Use Case**. L'obiettivo è stato rifinire e stabilizzare gli UC esistenti, con particolare riferimento a:

- **UC3 — Visualizza note:** coerenza dei dati restituiti, completamento dei campi necessari al frontend, gestione corretta del ruolo dell'autore;
- **UC5 — Versioni:** comportamento stabile e allineamento dell'interfaccia al flusso reale;
- **UC8 — Cerca nota:** bug su filtri e visibilità;

- **UC9 — Cartelle:** coerenza tra backend e frontend e ultimi fix sulla pagina cartelle;
- **UC10 — Modifica nota:** stabilità dell'update e refresh UI;
- **UC11 — Permessi:** riallineamento alle specifiche e correzioni sul modello di scambio dati.

Componenti trasversali consolidati:

- **Lock System** (coerenza timeout/unlock e casi limite);
- **Logging backend** (maggiore tracciabilità per debugging e auditing delle azioni);
- **Supporto operativo al debugging** (strumenti interni per ispezione MapDB).

Fase 2 — Discussione e suddivisione dei task

Product Owner (Lorenzo)

- validazione dei flussi principali (modifica, restore, cartelle, lock);
- verifica della completezza/coerenza delle informazioni mostrate in dashboard;
- segnalazione bug residui e validazione finale delle correzioni.

Scrum Master (Alessia)

- revisione tecnica delle PR e controllo qualità;
- verifica regressioni e coerenza con le specifiche;
- coordinamento test e gestione Kanban.

Developer 1 — Backend + Pull Request (Antonella)

PR 1 (stabilizzazione, strumenti e bug fixing principali)

- correzione di un bug relativo al comportamento dell'**autore** e completamento delle informazioni esposte dalla nota, includendo campi mancanti necessari alla UI e alle operazioni utente;
- introduzione di strumenti di supporto al debugging e alla verifica dei dati persistiti su MapDB, tramite la cartella **debug** con **NoteDBInspector** e **UserDBInspector**, in assenza di una GUI nativa equivalente a strumenti come MySQL Workbench;
- introduzione della cartella **logging** con **loggerActions**, e successiva integrazione nei service (refactoring) per garantire un **tracciamento strutturato delle azioni utente** e facilitare l'analisi dei flussi critici;
- consolidamento dei punti critici emersi dai test manuali e supporto all'integrazione backend–frontend;
- fix estetici/layout minori e correzione di incoerenze residue.

PR 2 (riallineamento permessi alle specifiche)

- riallineamento della gestione dei **permessi** alle specifiche, con refactoring del backend e introduzione di DTO dedicati, in particolare **PermessoRequest**, per rendere esplicita e stabile la richiesta/interpretazione del permesso lato API e garantire coerenza dei dati lato frontend;
- correzione delle ultime incoerenze di layout/design UI rilevate durante la fase di review.

Developer 2 — Frontend + Bug Fixing (Aurora)

- allineamento della UI ai dati aggiornati (NoteView/DTO e permessi) e stabilizzazione dei flussi in dashboard;
- correzioni su cartelle.html (selezione, refresh, coerenza colori/icone, aggiornamenti dinamici);
- fix su ricerca e filtri (note personali/condivise e casi limite);
- perfezionamenti UI minori (toast, badge, pulsanti) e correzione di dettagli di layout.

Sprint Backlog – Sprint 5

UC Raffinati	Product Backlog	Sprint Backlog	Task Terminati
UC1 – Registrazione UC2 – Login UC3 – Visualizza note UC4 – Crea nota UC5 – Visualizza versioni UC6 – Duplica nota UC7 – Rimuove utente UC8 – Cerca una nota UC9 – Aggiunge a cartella UC10 – Modifica nota UC11 – Modifica permessi UC12 – Elimina nota	Nessun nuovo UC: fase di stabilizzazione	Bug visualizzazione note Stabilizzazione NoteView Lock system (raffinamento) Fix cartelle (UI + backend) Logging backend Pulizia codice repository “pulita”	NoteView completa e coerente Fix dashboard visualizzazione Ricerca corretta (tutti i filtri) Cartelle aggiornate Lock system stabilizzato Logging migliorato Pulizia codice e repository

Output consegnati

Pull Request e incrementi consegnati

- **PR 1:** correzione bug sul ruolo dell'autore; completamento delle informazioni esposte dalla nota (campi mancanti necessari alla UI); introduzione cartella **debug** con NoteDBInspector e UserDBInspector per ispezione MapDB; introduzione cartella **logging** con loggerActions e integrazione nei service per tracciamento azioni; stabilizzazione di criticità emerse dai test manuali e supporto all'integrazione backend–frontend; fix estetici/layout minori.

- **PR 2:** refactoring permessi per riallineamento alle specifiche; introduzione del DTO **PermessoRequest** per la corretta interpretazione/trasmissione del permesso; correzione delle ultime incoerenze UI/layout.
- **PR 3:** pulizia finale dei file in data/ (MapDB notes.db / users.db) per consegna in stato “pulito”. Durante questa fase sono emersi alcuni **bug minori** (comportamenti non critici ma fastidiosi) che hanno richiesto ulteriori iterazioni di debugging e correzioni, soprattutto in ottica di coerenza tra frontend e backend nella versione finale. In parallelo, la suite di test è stata **aggiornata** per riflettere le ultime modifiche introdotte nei controller/service/DTO e, dove possibile, è stata anche **semplificata** per risultare più chiara e stabile (riduzione di casi ridondanti, test più mirati sui comportamenti principali).

Documentazione e standardizzazione finale

Durante lo Sprint 5 è stata **completata e consolidata la documentazione di progetto**, con una pianificazione esplicita dei diversi registri comunicativi in base al pubblico di riferimento. In particolare:

- **Diario del Progetto e Documentazione Sprint 0** sono stati mantenuti con stile **più formale e neutro**, orientato a rendere tracciabili decisioni, processo e risultati in modo chiaro e verificabile.
- Il **Manuale dello Sviluppatore** è stato organizzato per risultare **più leggibile e operativo**, con spiegazioni pratiche e sezioni pensate per favorire onboarding e manutenzione.
- Il **Manuale dell'Utente** è stato reso **marcatamente user-friendly**, con tono più guidato, una struttura “passo-passo” e un uso intenzionale e coerente di **emoji** per rendere la guida più immediata e allineata allo stile dell'applicazione.

Pulizia e rifinitura del codice per consegna

Nel corso degli sprint il team ha utilizzato **emoji nei log/commenti** come supporto rapido alla lettura e al riconoscimento dei flussi (debugging e tracciamento più immediato). In fase di consegna, è stata effettuata una **pulizia mirata** per rendere il progetto più uniforme e professionale, riducendo elementi ridondanti e consolidando uno stile più neutro dove opportuno.

In particolare, **dashboard.js** mantiene commenti e sezioni esplicative con l'obiettivo di **agevolare la lettura del codice** e la manutenzione (nei limiti di un file centralizzato e strutturalmente complesso), pur restando coerente con le scelte finali di stabilizzazione e chiarezza.

Stabilizzazione funzionale e tecnica

- **NoteView/contratti dati** più completi e coerenti, con informazioni necessarie sempre disponibili al frontend.
- **Permessi** riallineati alle specifiche e correttamente trasferiti tramite DTO (PermessoRequest).
- **Lock System** reso più prevedibile nei casi limite (timeout/unlock e integrazione con aggiornamenti UI).
- **Dashboard** più stabile nei refresh dinamici e nella gestione degli stati (inclusi filtri e operazioni successive).
- **Cartelle** consolidate con fix finali su pagina dedicata e coerenza aggiornamenti.
- **Logging backend** migliorato sia per debugging sia per tracciamento strutturato delle azioni utente tramite loggerActions.
- Introduzione di strumenti di **ispezione MapDB** (NoteDBInspector, UserDBInspector) per supportare debugging e verifiche dei dati persistiti, compensando l'assenza di un'interfaccia grafica nativa equivalente a tool come MySQL Workbench.

Sprint Review

Durante la Sprint Review, il team ha verificato la stabilità del sistema dopo le correzioni e i refactoring, controllando in particolare:

- coerenza della visualizzazione note e correttezza dei dati restituiti in tutti i flussi principali;
- stabilità del versionamento e del restore con aggiornamenti UI coerenti;
- comportamento del lock system nei casi critici;
- gestione cartelle e aggiornamenti dinamici;
- corretto comportamento dei **permessi** dopo il refactoring e l'introduzione di PermessoRequest.

È stata inoltre verificata l'efficacia degli strumenti introdotti per supporto operativo e debugging: gli inspector di MapDB (NoteDBInspector e UserDBInspector) per la lettura dei dati persistiti e il tracciamento delle azioni utente tramite loggerActions integrato nei service.

Il Product Owner ha confermato che, al termine dello sprint, il sistema risulta complessivamente stabile e utilizzabile, con riduzione significativa delle anomalie emerse negli sprint precedenti.

Sprint Retrospective

Problemi riscontrati

- alcuni bug residui risultavano difficili da riprodurre in modo deterministico e hanno richiesto iterazioni di test mirate;
la review delle specifiche ha evidenziato discrepanze che hanno richiesto refactoring non pianificati (in particolare su permessi e struttura dei dati scambiati);

- la complessità del frontend ha richiesto attenzione costante per evitare regressioni dopo i fix.

Azioni di miglioramento introdotte

- rafforzamento del logging per osservare e diagnosticare i flussi critici;
- consolidamento dell'approccio a DTO/NoteView come contratto stabile tra backend e frontend;
- adozione di strumenti interni (inspector MapDB e action logging) per ridurre il tempo di diagnosi dei bug e aumentare la trasparenza sui flussi applicativi;
- introduzione di una fase di review della documentazione in parallelo ai fix, per intercettare tempestivamente scostamenti dalle specifiche.

Regole consolidate

- Developer 1 apre PR
- Scrum Master approva
- Pair programming mantenuto
- Agenda condivisa sempre aggiornata

Conclusione

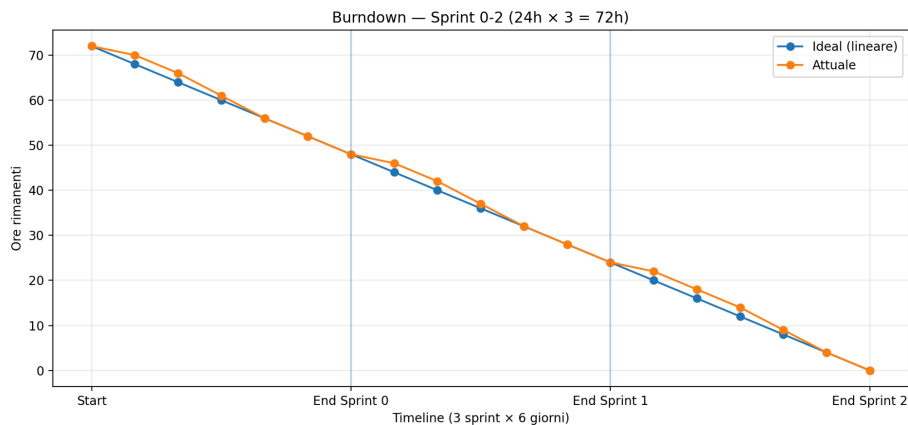
Lo Sprint 5 ha costituito una fase determinante di consolidamento, trasformando Nota Bene in un sistema **stabile, coerente e pronto per la consegna**. Pur non introducendo nuove funzionalità, lo sprint ha prodotto un incremento essenziale: correzione di bug emersi dai test e dalla review delle specifiche, completamento delle informazioni esposte dalla nota, riallineamento della gestione dei permessi tramite refactoring e introduzione di DTO dedicati (PermessoRequest), oltre a un rafforzamento degli strumenti di supporto al debugging e alla tracciabilità.

L'introduzione degli strumenti di ispezione MapDB (NoteDBInspector e UserDBInspector) e del tracciamento delle azioni utente tramite loggerActions ha migliorato la capacità del team di verificare i dati persistiti e diagnosticare più rapidamente i flussi critici, compensando l'assenza di una GUI dedicata per MapDB. L'avvio e la revisione della documentazione hanno inoltre contribuito a verificare la coerenza complessiva del progetto e a evidenziare tempestivamente eventuali discrepanze residue.

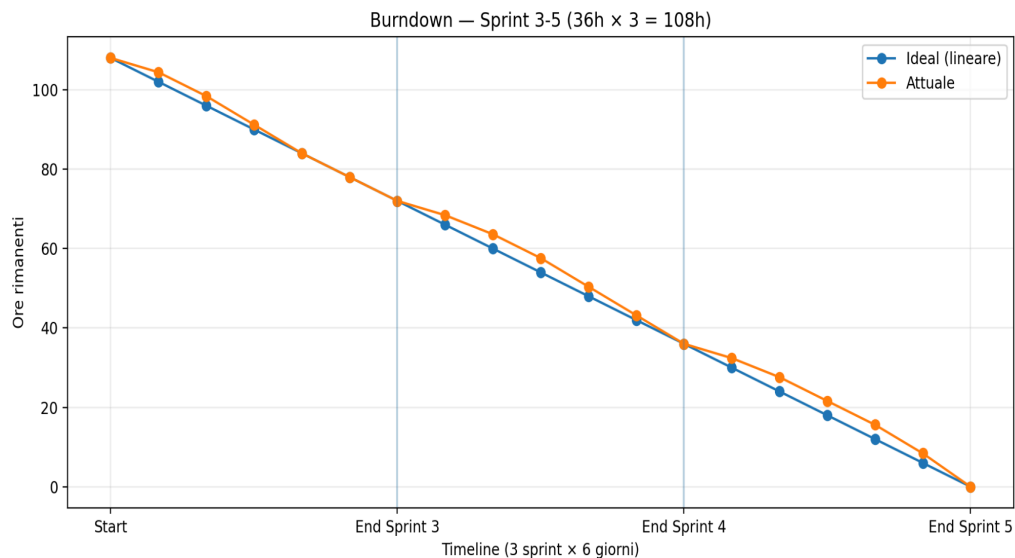
Con l'eventuale PR finale dedicata al database pulito, lo stato del sistema risulta pienamente compatibile con una consegna ordinata e verificabile, con un'applicazione completa dal punto di vista funzionale e adeguatamente consolidata dal punto di vista tecnico e documentale.

BurnDown Chart

Nel progetto, il team ha pianificato sprint settimanali e ha stimato l'impegno in **ore complessive di gruppo** per ciascuna iterazione, includendo attività di sviluppo, debugging, test, code review/Pull Request, riunioni e coordinamento. Nelle prime fasi (Sprint 0 – Sprint 2) la capacità è stata mantenuta pari a **24 ore/settimana**, coerente con l'obiettivo di costruire e consolidare un MVP e con un livello iniziale di complessità più contenuto.



A partire dallo **Sprint 3**, a fronte di un aumento significativo della complessità tecnica e organizzativa (integrazione della persistenza, definizione e validazione di regole di dominio più articolate, maggiore effort di integrazione frontend-backend e incremento del lavoro di test e rifinitura), il team ha **aggiustato la pianificazione** portando la capacity a **36 ore/settimana** negli sprint più impegnativi (Sprint 3 – Sprint 5). Il bilancio complessivo del progetto risulta, quindi, pari a **180 ore stimate** e **185 ore reali**, con un **delta di +2,8%**, indicativo di una stima complessivamente adeguata e di una gestione controllata dell'aumento di complessità nelle fasi finali.



Riflessione finale sul progetto e sul percorso di sprint

Il progetto Nota Bene, sviluppato tramite sprint settimanali, ha permesso al team di sperimentare in modo concreto un processo iterativo: partire da un MVP essenziale, estendere progressivamente il dominio funzionale e dedicare una parte rilevante del lavoro finale alla stabilizzazione, ai test e al riallineamento con le specifiche. Questo approccio ha reso visibili, sprint dopo sprint, sia i miglioramenti del prodotto sia le criticità che emergono quando la complessità aumenta, soprattutto durante l'integrazione tra componenti.

Dal punto di vista tecnologico, lo stack scelto si è rivelato adeguato al contesto del progetto: Spring Boot e Java 17 hanno favorito un backend strutturato (controller/service/repository) e un'evoluzione ordinata delle API, mentre MapDB ha consentito una persistenza semplice, evitando un overhead di configurazione tipico di un DBMS tradizionale. Al tempo stesso, l'assenza di strumenti grafici nativi ha reso più complessa la fase di debugging: per questo, l'introduzione di strumenti interni (inspector MapDB e logging delle azioni) e la standardizzazione dei DTO sono risultate scelte decisive per rendere più verificabili i flussi e più stabile la comunicazione frontend-backend.

Sul piano della collaborazione, la definizione di regole operative chiare (rotazione dei ruoli, pair programming, Pull Request in giorni concordati e comunicazione quotidiana) ha ridotto attriti organizzativi e facilitato l'apprendimento pratico del workflow. Le principali difficoltà hanno riguardato la crescita della complessità del frontend JavaScript, in particolare nella gestione dei numerosi stati UI e delle modali, e la gestione dei casi limite (lock, versioni, permessi, cartelle); tuttavia, proprio queste sfide ci hanno permesso di comprendere meglio concetti come regressioni, debito tecnico e importanza della fase di consolidamento. La fase di test manuali strutturati e di correzione incrementale ha avuto un ruolo determinante nel ridurre regressioni e nel rendere prevedibili i comportamenti dell'applicazione. Nel complesso, il progetto ha prodotto non solo un'applicazione completa e coerente rispetto alle specifiche, ma anche un percorso di crescita tecnica e organizzativa significativo per tutto il team.